

Exercise 6: Normalizing Flows

Dan Rosenbaum

Due: 30 June 2026

In this exercise we will use the same notebook we used in exercises 5.

<https://colab.research.google.com/drive/1l2vgJCpgHJFJ4HMkw6Ytsrm8EZnel9mo?usp=sharing>

The goal of this exercise is to implement a normalizing flow model, as presented in class.

In order to do that, use the same training process and model structure as implemented in the previous exercises. Follow the instructions below:

- Use the same continuous representation of the data as in exercise 5.
- The input image should be flattened and treated as a vector.
- The model needs to be composed of 10 normalizing flow (NF) layers, each using two MLP models with 5 linear layers and Relu activation functions. The last layer of the network should allow negative and positive values, so no relu should be used. Instead, a function like *tanh* can be used to force values to be between -1 and 1.
- Before each NF layer, split the input vector into two halves x_1 and x_2 . The next layer x' should be computed using the method we discussed in class, where its two halves x'_1 and x'_2 are constructed as follows:

$$\begin{aligned} x'_1 &= x_1 \\ x'_2 &= x_2 \odot \exp(s_\theta(x_1)) + m_\theta(x_1) \end{aligned}$$

Here s_θ and m_θ are the two networks of the layer. In each layer compute a different division to two halves (you can use alternating checkerboard type masking).

- Implement a *forward* function that maps input data points x into their latent representation z . The function needs to return the mapped latents z , and the log determinant of the Jacobian of the mapping.
- Implement an *inverse* function that maps latent vectors z into data samples x . The function needs to return the data samples.
- Implement a function *calc_nll* that returns the negative log-likelihood and will be used to train the model.
- Implement a function *sample* that will be used to generate samples.
- Use a Standard Gaussian distribution $\mathcal{N}(0, I)$ as the prior over z .

Assignments

1. Implement a normalizing flow model as described above and train it on the MNIST data. Train for about 30 epochs to get good results. You can update the architecture if you find something that works better.

Report the log-likelihood for the test set, curves showing the log-likelihood during training (for both training and validation data). After training, generate a few samples and show the results. You should be able to train models where most of the samples look like valid digits.

2. Show the transformation of samples from the latent distribution z to the data distribution. For 7 different samples show 11 images starting from the sample of the latent z , gradually transforming to samples at the output of each NF layer, up to the final output which is the sample of x . To do this you will need to change the output of the functions you implemented such that they output the values of intermediate layers. Show the results as a grid of 7 rows and 11 columns.

3. In this question you will generate latent traversals, like in exercise 5. Use the model that you trained, and choose two images of different digits from the dataset. For each image, compute the corresponding latent value z . This will be the 'representation' of the image. Now given the two latent representations z_1, z_2 , compute a linear path between them with a set of 10 points along the way. For each of those 10 points, z^i , compute the reconstruction sample x given z^i , and show the resulting images in order. You should see how the first digit transforms into the second digit. Repeat this for 3 different pairs of images.

You should submit a colab notebook that contains the text of the answers, figures and code for all the above questions. Add a few sentences that describe what you did and the results you got.